Writing Algorithms

What is an Algorithm?

An algorithm is simply a set of steps/instructions needed to complete a task. A recipe is an example of an algorithm. For instance you could define an algorithm to make a cup of tea. You start by filling the kettle, then place a tea bag in the cup and so on. When the word algorithm is used in computing, it defines the way in which a software task can be completed or the way a problem can be solved.

Writing algorithms is all about problem solving. After being given a problem to solve, programmers never begin programming straight away, they will first do the following:

- Think about the important parts of the problem (Abstraction),
- Break the problem down into smaller chunks (Decomposition),
- Think of the steps needed to solve each chunk (Algorithmic Thinking),
- Write down the steps – which in turn becomes the algorithm (Algorithmic Thinking).

Writing Algorithms

When writing algorithms, we could...

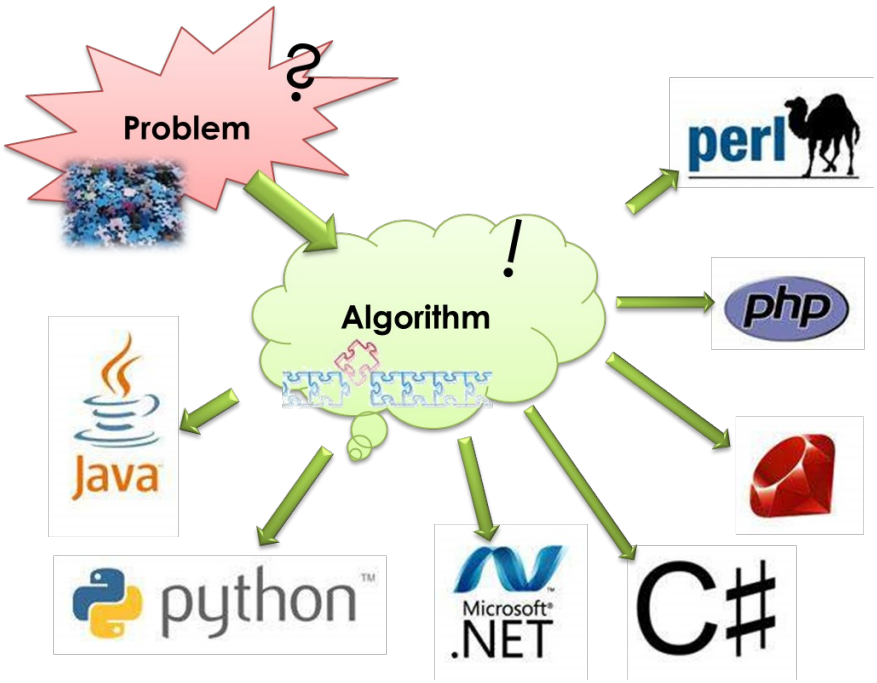
...write an algorithm in a specific language. However, doing this results in the following:

- Too time consuming
- Pointless – could just code
- Can't be taken by a programmer using a different language
- Too complex – need knowledge of syntax (code specific to a language)

...write an algorithm in everyday language. However, doing this results in the following:

- Too time consuming
- Open to interpretation – thus resulting in different outcomes

As a result, effective Algorithms should be independent of any language...



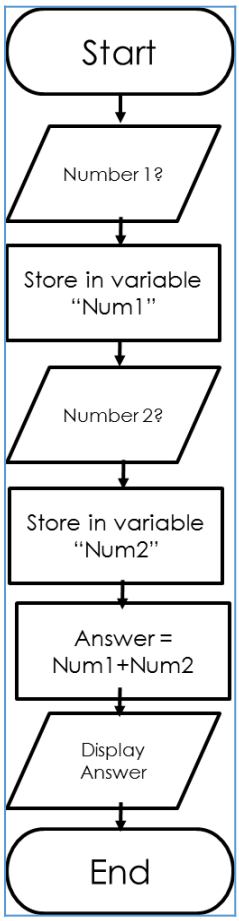
Computer Science UK Membership Site Licence: Do not share outside your centre.

There are therefore methods used to write algorithms so that they are succinct, accurate and easy to understand so that a programmer of any language could understand the steps required to solve a task. These methods are:

- Flowcharts
- Pseudocode

Examples of each method of algorithm writing:

Flowchart



Pseudocode

```
number1 = integer
number2 = integer
answer = integer

INPUT number1
INPUT number2
answer = number1 + number2

PRINT answer
```

Flowcharts

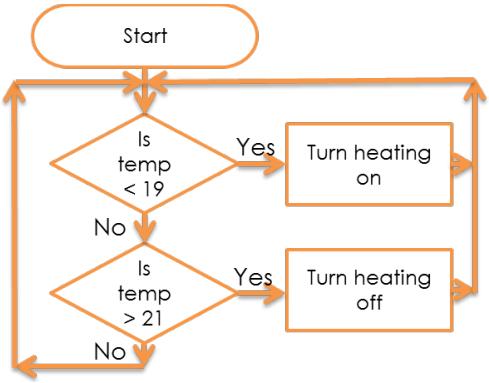
Flowcharts provide a visual representation of the logic of a program. They make use of a limited range of symbols to describe processes and arrows to show the order of instructions / program flow.

***Flowchart Example**:*

A central heating system will try to keep the temperature between 2 values (19 and 21)

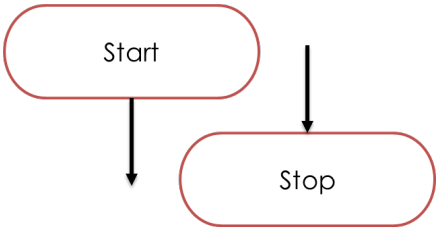
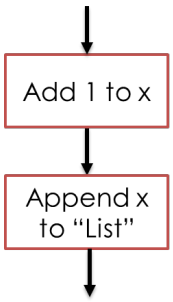
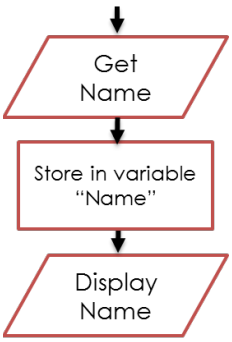
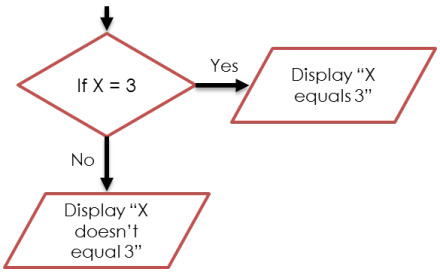
If the temperature falls below 19 it will turn the heating system on

If the temperature rises above 21 it will switch the heating system off.



Flowcharts and their symbols

Computer Science UK Membership Site Licence: Do not share outside your centre.

Representation	Explanation	Symbol
Start and Stop (Terminal) Symbols	All flow charts begin with a Start Symbol and at the end of the flow chart (or at various end points of the chart) we place a Stop Symbol. There are drawn as a rectangle with curved ends	
Process Symbols	Most of the time a flow chart will demonstrate the sequence of instructions to be carried out. Simple processes (like "Add 1 to x" or "append x to List") are shown using a standard rectangle.	
Input / Output Symbols	At times your program will most certainly ask the user for inputs and output values too. Inputs and Outputs (like "Name?" or "...display age") are shown using a parallelogram.	
Decision Symbols	At times your program will be programmed to make a decision based on certain conditions. Decisions (like "IF X = 3" or "WHILE Y > 3") are shown using a diamond.	

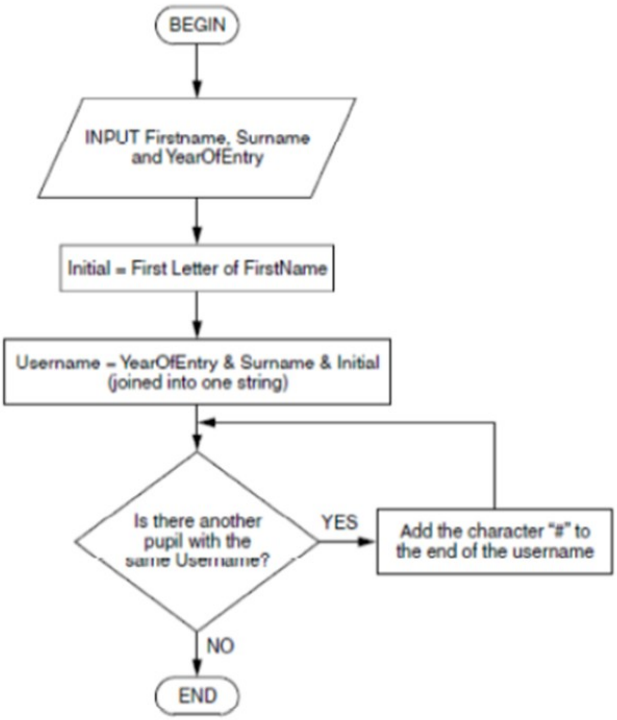
4 things you need to be able to do with flowcharts for your exam

- Understand them
- Correct them if they have errors,
- Complete them if they are incomplete
- Produce them after being given a problem to solve

Computer Science UK Membership Site Licence: Do not share outside your centre.

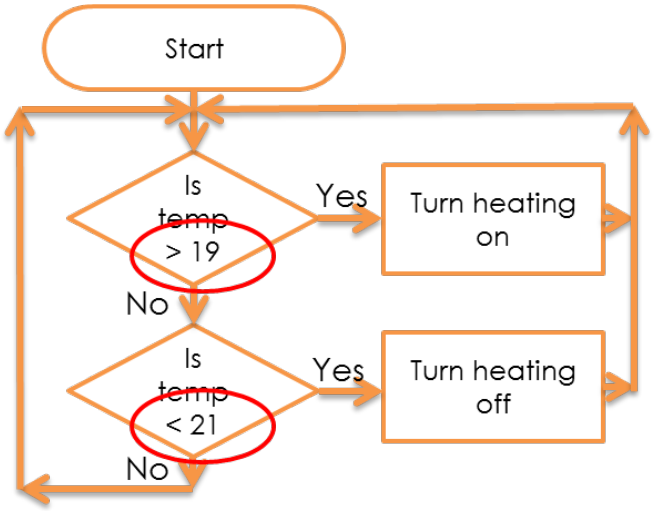
Tips on understanding / reading flowcharts:

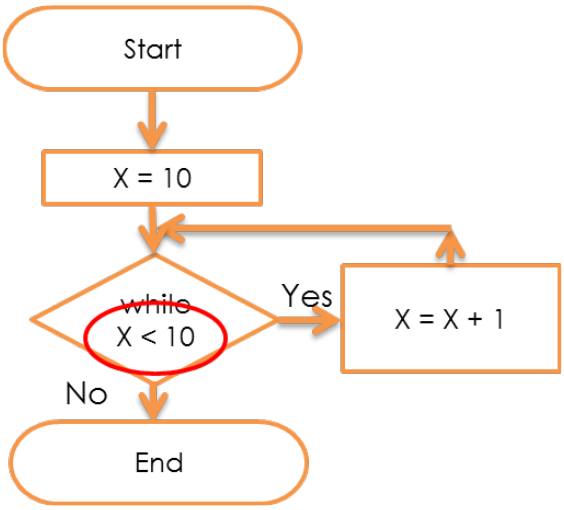
- Start from top and follow the arrows
- Remember what the symbols mean
- Read each symbol description
- Logically work out what the algorithms is doing.



Tips on correcting flowcharts:

Read the flowchart and look for decisions which don't make sense or iterations (loops) that you cannot enter of break out of.





Tips on completing flowcharts:

- Start from top and follow the arrows
- Remember what the symbols mean
- Read each symbol description
- Logically work out what the algorithms is doing.

...then use your knowledge of the algorithm so far and of the flow chart symbols and complete the flowchart.

Computer Science UK Membership Site Licence: Do not share outside your centre.

Pseudocode

Algorithms can be written in 'generic code' (which doesn't follow any particular syntax) as well as a flowchart. This 'generic code' is known as pseudocode. Pseudocode has keywords such as IF, ELSE and FOR and so mimics a programming language and therefore the logic is easy to follow and easy to turn into code. Although there are no STRICT STANDARDS for pseudocode there are some generally accepted keywords that you should be aware of.

Pseudocode Example:

- A central heating system will try to keep the temperature between 2 values (19 and 21)
- If the temperature falls below 19 it will turn the heating system on
- If the temperature rises above 21 it will switch the heating system off.

```
START
IF Temp < 19 THEN
        Heating ← On
ELSE
IF Temp > 21 THEN
        Heating ← Off
END
```

Pseudocode and commonly used keywords

Representation	Explanation	Symbol
Start and End "Key Words"	Pseudocode begin with a START and ends with END The algorithm goes in between.	**START** …… …… …… **END**
Process "Key Words"	Most of the time pseudocode will outline the logical sequence of instructions to be carried out. Simple processes will often use the key words shown below (like "CALCULATE X*2" or "INCREMENT X by 1)" You don't have to always use these words, for example the logic statements such as "Add 1 to x" or "append x to List") are fine too.	**Compute** **Calculate** **Determine** **Increment; ++ or +=** **Decrement; -- or -=**

Computer Science UK Membership Site Licence: Do not share outside your centre.

Input / Output "Key Words"	At times your program will most certainly ask the user for inputs and output values too. Inputs and Outputs (like "Name?" or "...display age") are indicated using the following words. Usually a programmer will choose one and stick with it throughout their algorithm.	INPUTS: OUTPUTS: **READ PRINT** **OBTAIN DISPLAY** **GET SHOW** **INPUT OUTPUT**
Variable Assignment "Key Words"	At times your program will assign values to variables. In pseudocode, this is done using the following key words.	**SET** **INIT** Sometimes an arrow is used instead e.g.: X ← 10 *This means SET X to 10*
Decision/Selection "Key Words"	At times your program will be programmed to make a decision based on certain conditions. Decisions (like "IF X = 3, THEN ...") are shown using, unsurprisingly, the following key words.	**IF** **THEN** **ELSE** **ELSE-IF** **ENDIF**
Loops / Iterations "Key Words"	Programs will often loop in places while certain conditions occur (condition controlled) or for a set number of times (count controlled). Loops use the following key words:	Count Controlled: **FOR** Condition Controlled: **WHILE / ENDWHILE** **REPEAT / UNTIL**

Pseudocode and their key words / statements in action...

Computer Science UK Membership Site Licence: Do not share outside your centre.

Notice:

Use of capitalised pseudocode key words/statements

Indentation

```
GET Name
FOR x = 1 TO 10
        DISPLAY "Your name is" = Name
END
```

```
GET Height
GET Base
Answer ← (Base * Height) / 2
DISPLAY "Right angled triangle area" = Answer
END
```

4 things you need to be able to do with pseudocode for your exam

- Understand them
- Correct them if they have errors,
- Complete them if they are incomplete
- Produce them after being given a problem to solve

Tips on understanding / reading pseudocode:

- Start from top and read each line in turn
- Remember what the key words mean
- Logically work out what the algorithms is doing.

```
START
WHILE TRUE
        X ← Temperature
        IF X < 19
                Heating ← TRUE
        ELSE-IF X > 21
                Heating ← FALSE
END
```

Tips on correcting pseudocode:

Read the pseudocode and look for decisions which don't make sense or iterations (loops) that you cannot enter of break out of.

```
START
WHILE TRUE
        X ← Temperature
        IF X > 19
                Heating ← TRUE
        ELSE-IF X < 21
                Heating ← FALSE
END
```

?

```
START
X ← 10
WHILE X < 10
        DO X ← X+1
END
```

Tips on completing pseudocode:

- Start from top and understand each line in turn
- Remember what the keywords mean
- Logically work out what the algorithms is doing

...then use your knowledge of the algorithm so far and of the pseudocode key words and complete the algorithm.

Tips on writing algorithms

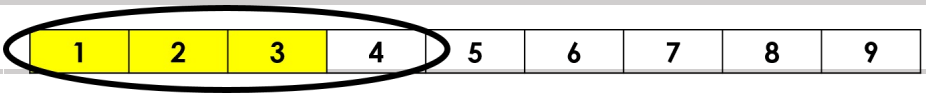
- Read the task to be solved

Computer Science UK Membership Site Licence: Do not share outside your centre.

- Read it again
- Highlight the important information
- Begin at the START of the problem
- Think logically in terms of '1 step at a time' and always think 'IF this were to happen, what should be the result?'

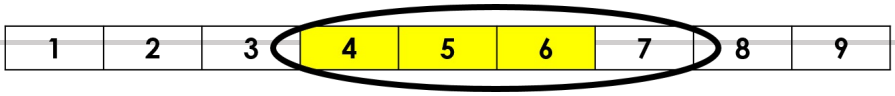
Questions *(The question zone you choose must either match your target grade or be higher!)*

Question Zone 1-3



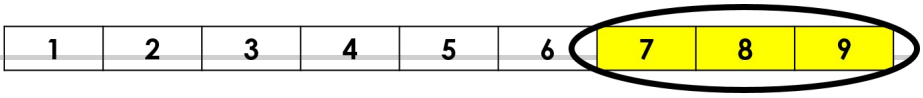
1. Define the term 'Algorithm'. [2]
2. Describe the difference between pseudocode and flowcharts. [2]
3. Write an algorithm in pseudocode, which asks for the user to input 2 numbers and then outputs their sum. [3]

Question Zone 4-6



1. Write an algorithm in pseudocode which asks for the user to input 2 numbers and then outputs their sum. [3]
2. Create a flowchart which shows the logic required to ask the user to input a word, check its length and output one of two messages depending on its length. If it is longer than 8 characters, "It's a long word" should be outputted, otherwise "It's a short word" should be outputted. [6]
3. "Effective algorithms are independent of any language". Explain the meaning of this statement [5]

Questions Zone 7-9



1. "Effective algorithms are independent of any language". Explain the meaning of this statement [5]
2. Create a flowchart which shows the logic required to repeatedly ask the user to enter a password until it matches the string "pa$$word". [5]
3. Discuss the two methods of writing algorithms and explain which you think is the most effective for an experienced software designer. [4]

Computer Science UK Membership Site Licence: Do not share outside your centre.

Checklist:

- Date and title, clearly presented
- Spelling & grammar checked
- Question numbers in the margin
- Handwriting neat & legible
- Punctuation / Capital letters

Keywords / Key Terms:

Algorithm: *A set of steps/instructions needed to complete a software task.*

Flowchart: *A graphical representation of an algorithm.*

Pseudocode: *A representation of an algorithm in which the algorithm resembles a simplified, generic programming language.*

Computer Science UK Membership Site Licence: Do not share outside your centre.

State/Identify/Give/Name: Simply label a diagram, fill out a table or write a few words
Describe: Describing is 'saying what you see' (E.G.: A computer will have a CPU, Primary and Secondary storage etc)
Explain: Explaining is 'saying WHY/HOW something is like that' (E.G.: A computer will have a CPU so that it can process all of the data the computer needs to perform a range of tasks. Primary and Secondary storage is needed because...)
Discuss: Discussing is 'looking at two sides of an issue, weighing up the two views and giving a conclusion'. Often these require a mini essay answer. (E.G.: New technology could be seen as being bad for the environment because..., but on the other hand, new technology has led to... In conclusion I believe that...)
Describe/Explain/Discuss **using examples**: Finally, if you are asked to give examples in any of these types of questions – YOU MUST **GIVE EXAMPLES**!

Stick answer sheet here

Computer Science UK Membership Site Licence: Do not share outside your centre.

Reflections **Score:** / **Percentage:** % **Grade:** **Progress** On / Above / Below

What Went

- I demonstrated a good level of understanding.
- I responded to the command words effectively.
- My answers were detailed / were written in depth.
- My work was well presented / legible.
- My answers effectively incorporated technical terminology.
- My responses were well structured / organised.
- My revision strategy was effective as I showed depth of understanding in my answers.
- My answers contained enough points / examples / explanations to achieve the marks available.

Even Better

- My answers need to be more accurate.
- I must respond correctly to the command words.
- My answers need more detail / greater depth.
- I must take greater care over my work / write neatly.
- I must incorporate key terminology into my answers.
- I must better organise my answers to improve its clarity.
- I need to improve my revision strategy, as I did not demonstrate a depth of understanding in my answers.
- My answers didn't contain enough points / examples / explanations to achieve the marks available.

Further thoughts:

_____

_____

_____

_____

Computer Science UK Membership Site Licence: Do not share outside your centre.

KEYWORDS

Computer Science UK Membership Site Licence: Do not share outside your centre.